

Mongo Manual

It is a document-based database. Stores data in the format of key value pair (JSON Format) called BSON

Records are called Documents.

Table -- Collection

Row -- Document

field -- Row

Database Commands -

show dbs -

shows all the database

use [DB_Name] -

to use or enter particular database

Eg - use intro

drop.[database_name] -

To delete a database

Eg - drop.intro()

Table Commands -

show Collections -

Used to return all the tables.

db.createCollection([DB_Name]) -

to create a table

Eg - `db.createCollection("intro")`

`db[table_name].insertOne({data})` -

To insert single data into table

Eg - `db.intro.insertOne({name : "abc" , age : 18});`

`db[table_name].insertMany([ {data1}, {data2}, {data3}])`

Eg - `db.intro.insertMany([ {name : "def" , age : 20}, {name : "ghi" , age : 22} , {name : "jkl" , age : 25} ])`

`db[table_name].find()` -

Fetch records from table

Eg - `db.intro.find()`

`db[collection_name].drop()` -

To delete a table

Eg - `db.intro.drop()`

`db[table_name].find({ [field] : [value to find]})` -

To extract or search specific value

Eg - `db.intro.find({ name : "Harsh"})`

`db[table_name].find({}, [column_name])` -

To extract records from specific column

Eg - `db.intro.find({}, "Name")`

`db[table_name].countDocuments({ "column_name" : {$exists : true} })` -

Used to extract records from specific column

Eg - `db.intro.countDocuments({ "Name" : {$exists : true} })`

```
db.[table_name].countDocuments({ [column_name] :  
[specific_value] })
```

Eg - db.intro.countDocuments({ Status : "Active" })

In case of Condition -

```
db.[table_name].find({ [field] , { [operator] :  
[value_to_find , value_to_find] }})
```

Eg - db.test.find({ name : {\$in : ["a" , "b"]} })

```
db.[table_name].updateOne( { [reference_field] :  
[reference_value] } , { $set [field_to_target] :  
[changed_value] } ) -
```

It is used to change the value of specific column

Eg - db.intro.updateOne({name : "Harsh"} , {\$set {name :
"Nish"}})

Group -

It is an aggregation pipeline that is used to group up things and then perform operations

Basic Syntax - db.[collection_name].aggregate[

\$group : {

_id : null , (when keeping null it means we are grouping all the documents in one)

total : { <operation_to_perform> : <field>

}

To sum up Specific field based on category provided in id

Syntax -> db.[collection_name].aggregate[

```
$group : {
```

```
  _id : <field_to_group> ,
```

```
  total : { <operation_to_perform> : <field
```

```
}
```

Example to print result in shell is json format -

```
let result = db.intro.updateOne({ name: "Harsh" }, { $set: {  
name: "Nish" } })
```

```
printjson(result)
```